

ARTIFICIAL INTELLIGENCE

Comprehensive Lecture Notes · Weeks 1–8 · Lectures 1–16

Instructor

Hafiz Muhammad Usman

WEEKS 1 – 2

Foundations

Intro to AI · Agents · Search Algorithms

WEEKS 3 – 7

Machine Learning

Supervised · Unsupervised · Metrics · Models

WEEK 8

Deep Learning & RL

Neural Networks · Perceptron · Q-Learning

8

WEEKS

16

LECTURES

5

ALGORITHMS

16

CASE STUDIES

Table of Contents

Week 1: Introduction to Artificial Intelligence

- L1 – Introduction & History of AI
- L2 – Components, Agents, Applications & Future of AI

Week 2: Problem Solving

- L3 – Problem Solving by Searching
- L4 – Uninformed Search – DFS & BFS

Week 3: Problem Solving & Machine Learning

- L5 – Informed Search – Greedy & A*
- L6 – Introduction to Machine Learning

Week 4: Machine Learning

- L7 – Types of Learning & Supervised Algorithms
- L8 – Classification & Naive Bayes

Week 5: Machine Learning

- L9 – Accuracy & Performance Metrics
- L10 – Linear Regression

Week 6: Machine Learning

- L11 – Logistic Regression
- L12 – K-Means Clustering

Week 7: Machine Learning

- L13 – Decision Trees & ID3
- L14 – Association Rules

Week 8: Neural Networks & Reinforcement Learning

- L15 – Neural Networks & Perceptron
- L16 – Reinforcement Learning

Week 1

Introduction to Artificial Intelligence

Lecture 1

Introduction & History of AI

1.1 What is Artificial Intelligence?

Artificial Intelligence (AI) is a branch of computer science that focuses on building systems capable of performing tasks that normally require human intelligence. These tasks include reasoning, learning, problem solving, perception, and language understanding. AI aims to simulate human cognitive functions in machines.

Definition: Artificial Intelligence

AI is the science and engineering of making intelligent machines, especially intelligent computer programs. It is related to the similar task of using computers to understand human intelligence, but AI does not have to confine itself to methods that are biologically observable. — John McCarthy, 1956

1.2 Four Schools of Thought

Category	Goal	Example
Thinking Humanly	Mimic human thought	Cognitive architectures
Acting Humanly	Pass as a human	Turing Test
Thinking Rationally	Use logic correctly	Theorem provers
Acting Rationally	Achieve best outcome	Autonomous agents

1.3 History of Artificial Intelligence

Era	Milestone	Key Figure
1943	McCulloch-Pitts neuron model introduced	McCulloch & Pitts
1950	Turing Test proposed in 'Computing Machinery and Intelligence'	Alan Turing
1956	Dartmouth Conference – 'AI' term coined	John McCarthy
1957	Perceptron invented	Frank Rosenblatt
1960s	LISP language developed; early expert systems	McCarthy / Minsky
1970s	First AI winter – funding cuts	—
1980s	Expert systems boom; second AI winter begins	Industry
1990s	Machine learning shifts focus; Deep Blue beats Kasparov	IBM

2012	Deep learning revolution – AlexNet wins ImageNet	Hinton et al.
2016	AlphaGo defeats world Go champion	DeepMind
2022+	Large Language Models (ChatGPT, Claude)	OpenAI / Anthropic

Example: Turing Test

In the Turing Test, a human judge engages in natural language conversations with both a human and a machine. If the judge cannot reliably tell which is which, the machine is said to have passed the test. Example: ELIZA (1966) was an early chatbot that fooled some people into thinking it was human.

Review Questions & Case Studies

A. Real-Time Review Questions

1. What is AI? Distinguish between 'Acting Rationally' and 'Thinking Rationally' with examples.
2. Describe the Turing Test. What are its limitations as a measure of intelligence?
3. Trace the major milestones in AI history from 1950 to 2022.
4. Why did AI winters occur? What factors triggered the revival of AI?
5. Compare human intelligence and artificial intelligence with at least three differences.

B. Case Study

Case Study – AI in Healthcare Diagnosis: IBM Watson for Oncology was deployed in several hospitals to assist doctors in diagnosing cancer. Watson analyzed millions of research papers, clinical trial data, and patient records to recommend treatment plans. In one study at Memorial Sloan Kettering Cancer Center, Watson agreed with oncologists in 99% of routine cancer cases and even identified treatment options doctors had missed.

Discussion Questions:

1. Which school of AI thought does IBM Watson represent?
2. What historical AI developments made Watson possible?
3. What ethical challenges arise when AI assists in medical decisions?

Lecture 2

Components of AI, Agents, Applications & Future

2.1 Core Components of AI

Machine Learning: Enables systems to learn from data without explicit programming.

Natural Language Processing (NLP): Allows machines to understand and generate human language.

Computer Vision: Enables machines to interpret visual information from the world.

Robotics: Physical machines that perceive and act in the real world.

Expert Systems: Systems that encode human expert knowledge in rule-based form.

Knowledge Representation: Methods for storing and reasoning about knowledge.

2.2 What is an AI Agent?

Definition: AI Agent

An AI agent is anything that can perceive its environment through sensors and act upon that environment through actuators. Formally: Agent = Perception + Action + Goal + Environment

2.3 PEAS Framework for Agents

Every agent is described using the PEAS framework:

Component	Description	Self-Driving Car Example
P – Performance	Measure of success	Safety, speed, fuel efficiency
E – Environment	Where agent operates	Roads, traffic, pedestrians
A – Actuators	How agent acts	Steering, brakes, throttle
S – Sensors	How agent perceives	Cameras, GPS, LiDAR

2.4 Types of AI Agents

Simple Reflex Agent: Acts only on current perception. No memory. Example: Thermostat.

Model-Based Reflex Agent: Maintains internal state to handle partially observable environments.

Goal-Based Agent: Acts to achieve specific goals. Example: Navigation system.

Utility-Based Agent: Selects actions that maximize a utility function. Example: Game AI.

Learning Agent: Improves performance over time through experience. Example: Spam filter.

2.5 Real-World Applications of AI

Domain	Application	Example
Healthcare	Disease diagnosis, drug discovery	IBM Watson, AlphaFold
Finance	Fraud detection, algorithmic trading	PayPal fraud detection

Education	Adaptive learning, automated grading	Duolingo, Coursera AI
Transportation	Self-driving cars, route optimization	Tesla Autopilot, Uber
Agriculture	Crop monitoring, yield prediction	John Deere AI tractors
Customer Service	Chatbots, virtual assistants	Siri, Alexa, Google Assistant

2.6 Future of AI

Key future trends include: **Artificial General Intelligence (AGI)** – machines with human-level general reasoning; **Explainable AI (XAI)** – AI that explains its own decisions; **Edge AI** – running AI on low-power devices; **AI in Quantum Computing** – exponentially faster problem solving; **AI Ethics & Regulation** – ensuring fairness, transparency, and accountability in AI systems.

Review Questions & Case Studies

A. Real-Time Review Questions

1. Define an AI agent. Explain the PEAS framework with a smart home assistant example.
2. Compare Simple Reflex Agent and Learning Agent with a real-world scenario.
3. List five applications of AI in daily life and explain how AI is used in each.
4. What is Natural Language Processing? Give three real-world applications.
5. Discuss two ethical challenges that AI poses in the near future.

B. Case Study

Case Study – AI in Agriculture (Precision Farming): John Deere has integrated AI into its tractors and farming equipment. Using computer vision and ML, these tractors can identify individual plants, distinguish crops from weeds, and apply herbicide only where needed — reducing chemical use by up to 90%. Sensors collect soil data and drones map crop health across large fields in real time.

Discussion Questions:

1. Which AI components are involved in this precision farming system?
2. Describe this system using the PEAS framework.
3. What type of AI agent best describes this farming robot?

Week 2

Problem Solving

Lecture 3

Problem Solving by Searching

3.1 Problem Formulation

Problem solving in AI involves finding a sequence of actions that leads from an initial state to a goal state. This process is called **search**. To solve a problem by searching, we must formally define it.

Definition: Search Problem

A search problem consists of: (1) Initial State – where the agent starts, (2) Actions – possible moves from any state, (3) Transition Model – result of each action, (4) Goal Test – checks if current state is the goal, (5) Path Cost – cost of each path.

3.2 State Space Representation

Example: 8-Puzzle Problem

The 8-puzzle has 9 tiles on a 3x3 grid. One tile is blank. Initial state: [2,8,3,1,6,4,7,_,5]. Goal state: [1,2,3,8,_,4,7,6,5]. Actions: move blank UP, DOWN, LEFT, RIGHT. Path cost: 1 per move. Total states: $9!/2 = 181,440$. This is a classic search problem.

3.3 Search Tree vs State Space

A **state space** is a graph of all possible states and transitions. A **search tree** is generated by expanding nodes from the root (initial state). Nodes contain: State, Parent, Action, Path Cost, Depth.

Property	State Space	Search Tree
Structure	Graph (may have cycles)	Tree (no cycles by design)
Memory	Fixed size	Can grow exponentially
Purpose	Defines the problem	Used to find the solution

3.4 Measuring Search Performance

Completeness: Will the algorithm always find a solution if one exists?

Optimality: Does the algorithm find the least-cost solution?

Time Complexity: How many nodes are generated/expanded?

Space Complexity: How much memory is needed?

Review Questions & Case Studies

A. Real-Time Review Questions

1. Define a 'search problem'. List and explain its five components with an example.
2. What is the difference between a state space and a search tree?
3. Formulate the Romanian map problem: identify initial state, actions, goal, and path cost.
4. What are the four criteria for evaluating a search algorithm?
5. Represent the Water-Jug problem (3L and 5L jugs, goal: 4L) as a state space search.

B. Case Study

Case Study – Google Maps Route Planning: Google Maps uses search algorithms to find optimal routes between two locations. The state space includes intersections (nodes) and roads (edges with costs based on distance and traffic). When you enter a destination, the app searches through millions of possible paths to find the fastest or shortest route in real time.

Discussion Questions:

1. Map the Google Maps problem to the five components of a search problem.
2. What 'path cost' would Google Maps minimize: distance, time, or fuel?
3. Is Google Maps' search complete and optimal? Justify your answer.

Lecture

4

Uninformed Search – DFS & BFS

4.1 Uninformed (Blind) Search

Uninformed search algorithms have no additional information about states beyond what is provided in the problem definition. They search systematically through the state space without any knowledge of how close they are to the goal.

4.2 Breadth-First Search (BFS)

Definition: BFS

BFS expands the shallowest (least deep) unexpanded node first. It uses a FIFO queue. BFS is complete and optimal (for unit cost). Time: $O(b^d)$, Space: $O(b^d)$ where b = branching factor, d = depth of shallowest goal.

Example: BFS on a Tree

Tree: A connects to B,C. B connects to D,E. C connects to F,G. BFS order: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$. Level 0: {A}, Level 1: {B,C}, Level 2: {D,E,F,G}. If goal = G, BFS explores 7 nodes.

BFS Python Implementation

```
from collections import deque

def bfs(graph, start, goal):
    queue = deque([start])      # Queue of paths
    visited = set([start])

    while queue:
        path = queue.popleft()
        node = path[-1]

        if node == goal:
            return path          # Return full path to goal

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                new_path = path + [neighbor]
                queue.append(new_path)

    return None                  # No path found

# Example usage
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
}
```

```
'D': [], 'E': [], 'F': [], 'G': []
}
print(bfs(graph, 'A', 'G'))    # Output: ['A', 'C', 'G']
```

4.3 Depth-First Search (DFS)

Definition: DFS

DFS expands the deepest unexpanded node first. It uses a LIFO stack (or recursion). DFS is NOT complete (may loop in infinite graphs) and NOT optimal. Time: $O(b^m)$, Space: $O(bm)$ where m = maximum depth. Space-efficient.

```
def dfs(graph, start, goal, visited=None, path=None):
    if visited is None:
        visited = set()
    if path is None:
        path = []

    visited.add(start)
    path = path + [start]

    if start == goal:
        return path

    for neighbor in graph[start]:
        if neighbor not in visited:
            result = dfs(graph, neighbor, goal, visited, path)
            if result:
                return result

    return None

# Example usage
graph = {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F', 'G'],
        'D': [], 'E': [], 'F': [], 'G': []}
print(dfs(graph, 'A', 'G'))    # Output: ['A', 'C', 'G']
```

4.4 BFS vs DFS Comparison

Property	BFS	DFS
Data Structure	Queue (FIFO)	Stack (LIFO)
Completeness	Yes (finite)	No (may loop)
Optimality	Yes (unit cost)	No
Time Complexity	$O(b^d)$	$O(b^m)$
Space Complexity	$O(b^d)$	$O(bm)$
Best Use Case	Shortest path	Memory limited

Review Questions & Case Studies

A. Real-Time Review Questions

1. Explain BFS with an example. Under what conditions is BFS optimal?
2. Implement DFS on the following graph and trace the visited order: $A \rightarrow B, C$; $B \rightarrow D, E$; $C \rightarrow F$.
3. Compare BFS and DFS on time complexity, space complexity, completeness, and optimality.
4. What is the branching factor? If $b=3$ and $d=4$, how many nodes does BFS explore?
5. When would you prefer DFS over BFS in a real-world scenario?

B. Case Study

Case Study – Web Crawler using BFS: Search engines like Google use BFS-like algorithms to crawl the web. Starting from a seed URL, the crawler follows all links on that page (Level 1), then all links on those pages (Level 2), and so on. BFS ensures pages are discovered in order of their 'distance' from the seed — important for ranking relevance.

Discussion Questions:

1. Why is BFS preferred over DFS for web crawling?
2. What would be the 'branching factor' for a web page with 50 links?
3. How does a crawler avoid visiting the same URL twice?

Week 3

Problem Solving & Introduction to Machine Learning

Lecture 5

Informed Search – Greedy & A*

5.1 Informed (Heuristic) Search

Informed search uses problem-specific knowledge (a heuristic function) to guide the search towards the goal more efficiently than uninformed methods. A **heuristic function $h(n)$** estimates the cost from node n to the goal.

Definition: Heuristic Function $h(n)$

$h(n)$ = estimated cost from node n to the goal. Admissible heuristic: $h(n)$ never overestimates the true cost. Example (8-puzzle): $h(n)$ = number of misplaced tiles (always $\leq$ actual moves needed).

5.2 Greedy Best-First Search

Greedy search expands the node that appears closest to the goal based solely on $h(n)$. It is fast but NOT optimal — it may miss shorter paths.

Example: Romania Map (Greedy)

Goal: Bucharest from Arad. Heuristic h = straight-line distance to Bucharest. Greedy picks: Arad(366) $\rightarrow$ Sibiu(253) $\rightarrow$ Fagaras(176) $\rightarrow$ Bucharest(0). Path cost = 140+99+211 = 450 km. Optimal path costs 418 km. Greedy is NOT optimal here.

5.3 A* Search

Definition: A* Search

A* uses $f(n) = g(n) + h(n)$ where $g(n)$ = actual cost from start to n , $h(n)$ = estimated cost from n to goal. A* is both complete and optimal when h is admissible.

```
import heapq

def astar(graph, start, goal, heuristic):
    # Priority queue: (f_cost, node, path, g_cost)
    open_set = [(heuristic[start], start, [start], 0)]
    visited = {}

    while open_set:
        f, node, path, g = heapq.heappop(open_set)

        if node in visited:
            continue
        visited[node] = g
```

```

    if node == goal:
        return path, g    # Return path and total cost

    for neighbor, cost in graph[node].items():
        if neighbor not in visited:
            g_new = g + cost
            f_new = g_new + heuristic[neighbor]
            heapq.heappush(open_set, (f_new, neighbor, path+[neighbor], g_new))

    return None, float('inf')

# Romania simplified example
graph = {
    'Arad':  {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
    'Sibiu':  {'Arad': 140, 'Fagaras': 99, 'Rimnicu': 80},
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Zerind':  {'Arad': 75},
    'Timisoara': {'Arad': 118},
    'Rimnicu': {'Sibiu': 80},
    'Bucharest': {'Fagaras': 211}
}

heuristic = {'Arad': 366, 'Zerind': 374, 'Timisoara': 329, 'Sibiu': 253,
             'Fagaras': 176, 'Rimnicu': 193, 'Bucharest': 0}

path, cost = astar(graph, 'Arad', 'Bucharest', heuristic)
print(f"Path: {path}, Cost: {cost}")
# Output: Path: ['Arad', 'Sibiu', 'Fagaras', 'Bucharest'], Cost: 450

```

5.4 Comparing Search Algorithms

Algorithm	Complete	Optimal	Time	Space
BFS	Yes	Yes*	$O(b^d)$	$O(b^d)$
DFS	No	No	$O(b^m)$	$O(bm)$
Greedy	No	No	$O(b^m)$	$O(b^m)$
A*	Yes	Yes	$O(b^d)$	$O(b^d)$

Review Questions & Case Studies

A. Real-Time Review Questions

1. What is a heuristic function? What does 'admissible' mean in the context of A*?
2. Explain A* search with $f(n) = g(n) + h(n)$. Why is A* optimal when h is admissible?
3. Compare Greedy Best-First and A* search. When does Greedy fail to find optimal path?
4. Trace A* on the Romania map from Arad to Bucharest step by step.
5. Design an admissible heuristic for the 8-puzzle problem and justify why it is admissible.

B. Case Study

Case Study – GPS Navigation with A*: Modern GPS systems like Waze use A* or its variants. The heuristic $h(n)$ is the straight-line distance from the current intersection to the destination. $g(n)$ accumulates actual travel time/distance. The algorithm explores fewer nodes than BFS, making real-time routing feasible even in cities with millions of intersections.

Discussion Questions:

1. Why is straight-line distance an admissible heuristic for road routing?
2. How would traffic data be incorporated into $g(n)$?
3. Would Greedy search work for GPS? What problems might arise?

Lecture 6

Introduction to Machine Learning

6.1 What is Machine Learning?

Definition: Machine Learning

Machine Learning (ML) is a subset of AI that gives systems the ability to automatically learn and improve from experience without being explicitly programmed. ML focuses on building algorithms that can access data and use it to learn for themselves. — Tom Mitchell: 'A computer program is said to learn from experience E with respect to task T and performance measure P , if its performance on T improves with E .'

6.2 Traditional Programming vs Machine Learning

Aspect	Traditional Programming	Machine Learning
Input	Data + Rules	Data + Output (Labels)
Output	Answers	Rules / Model
Approach	Logic-driven	Data-driven
Flexibility	Low (rigid rules)	High (adapts to data)
Example	if temp > 100: boil	learns when water boils from examples

6.3 The Machine Learning Pipeline

1. Data Collection → 2. Data Preprocessing → 3. Feature Engineering → 4. Model Selection → 5. Model Training → 6. Model Evaluation → 7. Hyperparameter Tuning → 8. Deployment

6.4 Key Terminology

Feature (X): An input variable used to make predictions. E.g., age, income, temperature.

Label (y): The output variable we want to predict. E.g., price, category, spam/not-spam.

Training Set: Data used to train (fit) the model — typically 70–80% of dataset.

Test Set: Data used to evaluate model performance — typically 20–30%.

Model: A mathematical function that maps inputs X to outputs y .

Overfitting: Model learns training data too well, fails on new data (high variance).

Underfitting: Model is too simple, fails even on training data (high bias).

Review Questions & Case Studies

A. Real-Time Review Questions

1. Define Machine Learning using Tom Mitchell's formal definition with a real-world example.
2. Compare traditional programming and machine learning using a spam detection example.

3. List and explain each step of the ML pipeline with a house price prediction scenario.
4. What is overfitting? How can it be detected and prevented?
5. What is the difference between a feature and a label? Give five examples of each.

B. Case Study

Case Study – Netflix Recommendation System: Netflix uses ML to recommend shows to 200+ million subscribers. The system analyzes viewing history (features: genres watched, time of day, completion rate, ratings) to predict what a user will enjoy next (label: probability of watching). Without ML, creating explicit rules for every user's taste would be impossible. The model improves as more data is collected.

Discussion Questions:

1. Identify at least five features Netflix's recommendation model might use.
2. Which step of the ML pipeline would be hardest for Netflix? Why?
3. What problems could arise if the model overfits on a user's viewing history?

Week 4 Machine Learning – Types & Supervised Learning

Lecture 7 Types of Learning & Supervised Learning Algorithms

7.1 Types of Machine Learning

Type	Description	Examples
Supervised	Learns from labeled data	Classification, Regression
Unsupervised	Finds patterns in unlabeled data	Clustering, Dimensionality Reduction
Semi-Supervised	Mix of labeled & unlabeled	Self-training models
Reinforcement	Learns via reward/penalty	Game AI, Robotics

7.2 Supervised Learning in Detail

In supervised learning, the algorithm learns from a labeled training dataset. Each training example is a pair (X, y) where X is the feature vector and y is the label. The goal is to learn a function f such that $f(X) \approx y$ for unseen data.

7.3 Regression vs Classification

Aspect	Regression	Classification
Output	Continuous number	Discrete category
Example	Predict house price	Predict email: spam/ham
Algorithms	Linear Regression	Logistic Regression, SVM, Naive Bayes
Metric	MSE, RMSE, R^2	Accuracy, Precision, Recall, F1

7.4 Learning Models Overview

Linear Models: Linear Regression, Logistic Regression – fast, interpretable, work best on linearly separable data.

Tree Models: Decision Trees, Random Forest – handle non-linear data, easy to interpret.

Probabilistic Models: Naive Bayes – fast, good for text classification, assumes feature independence.

Distance-Based: K-Nearest Neighbors – classifies by majority vote of k nearest training examples.

Kernel Methods: Support Vector Machines – find optimal hyperplane to separate classes.

Ensemble Methods: Bagging, Boosting, Random Forest – combine multiple models for better performance.

Review Questions & Case Studies

A. Real-Time Review Questions

1. List and explain the four types of machine learning with one real-world example each.
2. What is supervised learning? How does it differ from unsupervised learning?
3. Distinguish between regression and classification. Give two examples of each.
4. What is a training set and a test set? Why should they never overlap?
5. Explain overfitting and underfitting using a diagram or analogy.

B. Case Study

Case Study – Credit Card Fraud Detection (Supervised Learning): Banks like Visa and Mastercard use supervised learning to detect fraudulent transactions. Features include: transaction amount, location, time, merchant category, spending pattern deviation. Label: fraud (1) or legitimate (0). The model is trained on millions of historical transactions. When a new transaction occurs, the model predicts in milliseconds whether it is fraudulent.

Discussion Questions:

1. Is fraud detection a regression or classification problem? Justify.
2. What challenges arise with imbalanced datasets (99.9% legitimate, 0.1% fraud)?
3. Which supervised learning model would you choose and why?

Lecture 8

Classification & Naive Bayes

8.1 What is Classification?

Definition: Classification

Classification is a supervised learning task where the model learns to assign input data to one of several predefined categories (classes). Binary classification has 2 classes; multi-class has 3+. Output is a discrete class label.

8.2 Naive Bayes Classifier

Naive Bayes is a probabilistic classifier based on **Bayes' Theorem** with a 'naive' assumption that all features are conditionally independent given the class.

Definition: Bayes' Theorem

$P(C|X) = [P(X|C) \times P(C)] / P(X)$ where: $P(C|X)$ = posterior probability of class C given features X, $P(X|C)$ = likelihood of features X given class C, $P(C)$ = prior probability of class C, $P(X)$ = probability of features X (normalizer).

Example: Email Spam Detection

Training data: 100 emails (60 spam, 40 ham). Word 'free' appears in 45 spam and 5 ham emails. New email contains word 'free'. $P(\text{spam}) = 60/100 = 0.6$, $P(\text{free}|\text{spam}) = 45/60 = 0.75$, $P(\text{free}|\text{ham}) = 5/40 = 0.125$. $P(\text{spam}|\text{free}) \propto 0.75 \times 0.6 = 0.45$. $P(\text{ham}|\text{free}) \propto 0.125 \times 0.4 = 0.05$. Since $0.45 > 0.05 \rightarrow$ classify as SPAM.

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Sample data
emails = [
    "free money win prize", "free offer discount",
    "meeting tomorrow agenda", "project deadline next week",
    "win lottery free cash", "hello how are you doing"
]
labels = [1, 1, 0, 0, 1, 0] # 1=spam, 0=ham

# Convert text to feature vectors
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(emails)

# Split into train and test
X_train, X_test, y_train, y_test = train_test_split(
    X, labels, test_size=0.3, random_state=42)
```

```
# Train Naive Bayes
model = MultinomialNB()
model.fit(X_train, y_train)

# Predict and evaluate
y_pred = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")

# Classify a new email
new_email = ["win free prize lottery"]
X_new = vectorizer.transform(new_email)
prediction = model.predict(X_new)
print("Spam" if prediction[0] == 1 else "Ham")
```

8.3 Types of Naive Bayes

Type	Use Case	Feature Distribution
Gaussian NB	Continuous features	Normal distribution
Multinomial NB	Text classification	Word counts
Bernoulli NB	Binary features	0 or 1 (present/absent)

Review Questions & Case Studies

A. Real-Time Review Questions

1. State Bayes' Theorem. Explain each component (prior, likelihood, posterior) with examples.
2. Apply Naive Bayes to classify the following email as spam or ham (show calculations).
3. What is the 'naive' assumption in Naive Bayes? When does it fail?
4. Compare Gaussian, Multinomial, and Bernoulli Naive Bayes — when to use each?
5. What is Laplace smoothing? Why is it needed in Naive Bayes?

B. Case Study

Case Study – Medical Diagnosis with Naive Bayes: Doctors at a hospital want to build a system to classify patients as having diabetes or not based on: glucose level, BMI, blood pressure, age, and family history. Naive Bayes calculates the probability of diabetes given these symptoms independently, combining them into a final classification. Despite the naive independence assumption, it often performs surprisingly well in medical contexts.

Discussion Questions:

1. Which type of Naive Bayes (Gaussian/Multinomial/Bernoulli) would you use here?
2. What prior probability $P(\text{diabetes})$ would you use for the general population?
3. What are the ethical implications of a false negative in this scenario?

Week 5

Machine Learning – Metrics & Linear Regression

Lecture 9

Accuracy & Performance Metrics

9.1 Why Accuracy Alone Is Not Enough

In a cancer detection dataset, 99% of patients are healthy and 1% have cancer. A model that always predicts 'healthy' achieves 99% accuracy — but it misses ALL cancer cases! This shows why we need more nuanced metrics.

9.2 Confusion Matrix

Definition: Confusion Matrix

A confusion matrix summarizes the performance of a classifier: TP (True Positive) = correctly predicted positive; TN (True Negative) = correctly predicted negative; FP (False Positive) = predicted positive, actually negative (Type I error); FN (False Negative) = predicted negative, actually positive (Type II error).

	Predicted: Positive	Predicted: Negative
Actual: Positive	TP (True Positive)	FN (False Negative)
Actual: Negative	FP (False Positive)	TN (True Negative)

9.3 Key Performance Metrics

Metric	Formula	Best When
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Balanced classes
Precision	$TP/(TP+FP)$	FP is costly (spam filter)
Recall	$TP/(TP+FN)$	FN is costly (cancer detection)
F1-Score	$2 \times (P \times R) / (P + R)$	Imbalanced classes
Specificity	$TN/(TN+FP)$	Ruling out negatives

```
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             precision_score, recall_score, f1_score,
                             classification_report)

import numpy as np

# Actual and predicted labels
y_true = [1,1,1,0,0,0,1,0,1,0]
```

```
y_pred = [1,0,1,0,1,0,1,0,0,0]

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
print("Confusion Matrix:")
print(cm)

# Individual Metrics
print(f"Accuracy : {accuracy_score(y_true, y_pred):.2f}")
print(f"Precision: {precision_score(y_true, y_pred):.2f}")
print(f"Recall   : {recall_score(y_true, y_pred):.2f}")
print(f"F1-Score : {f1_score(y_true, y_pred):.2f}")

# Full Report
print(classification_report(y_true, y_pred,
                           target_names=['Not Spam', 'Spam']))
```

9.4 ROC Curve and AUC

The **ROC (Receiver Operating Characteristic)** curve plots True Positive Rate vs False Positive Rate at various classification thresholds. **AUC (Area Under Curve)** measures the overall ability of the model to discriminate between classes. AUC = 1.0 is perfect; AUC = 0.5 is random.

Review Questions & Case Studies

A. Real-Time Review Questions

1. Explain the four components of a confusion matrix with a medical diagnosis example.
2. Calculate precision, recall, and F1-score for: TP=80, FP=20, FN=15, TN=885.
3. When is recall more important than precision? Give a real-world example.
4. What does AUC = 0.85 mean? How does it differ from accuracy?
5. You have an imbalanced dataset (95% class A, 5% class B). Which metric should you use?

B. Case Study

Case Study – COVID-19 Test Evaluation: A rapid COVID-19 test was evaluated on 1000 patients. Results: TP=90 (correctly detected COVID), FP=50 (false alarm), FN=10 (missed COVID), TN=850 (correctly cleared). The health ministry needed to decide: should they optimize for precision (reduce false alarms) or recall (reduce missed cases)?

Discussion Questions:

1. Calculate accuracy, precision, recall, and F1 for this test.
2. For a pandemic screening tool, which metric is more critical — precision or recall?
3. What is the cost of a false negative in this medical context?

Lecture 10

Supervised Learning – Linear Regression

10.1 What is Linear Regression?

Definition: Linear Regression

Linear Regression models the relationship between a dependent variable y and one or more independent variables X by fitting a linear equation: $y = mx + b$ (simple) or $y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$ (multiple). It predicts a continuous output.

Example: House Price Prediction

Features: house size (sqft), number of rooms, location score. Label: price in PKR. Simple model: Price = 5000 × Size + 200000. For a 1000 sqft house: Price = 5000 × 1000 + 200000 = PKR 5,200,000.

10.2 Cost Function – Mean Squared Error

The model learns by minimizing the **Mean Squared Error (MSE)**: the average squared difference between predicted and actual values. $MSE = (1/n) \times \sum (y_i - \hat{y}_i)^2$. Lower MSE means better fit.

10.3 Gradient Descent

Gradient Descent is the optimization algorithm used to minimize the cost function. It iteratively updates weights in the direction of steepest descent: $w = w - \alpha \times (\partial MSE / \partial w)$, where α is the learning rate.

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

# Generate sample data: house size vs price
np.random.seed(42)
sizes = np.random.randint(500, 3000, 100).reshape(-1, 1)
prices = 4500 * sizes.flatten() + 150000 + np.random.randn(100) * 50000

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    sizes, prices, test_size=0.2, random_state=42)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
```

```
# Evaluate
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Coefficient (slope) : {model.coef_[0]:.2f}")
print(f"Intercept : {model.intercept_: .2f}")
print(f"MSE : {mse:.2f}")
print(f"R-squared : {r2:.4f}")

# Predict for a new house
new_size = np.array([[1200]])
predicted_price = model.predict(new_size)
print(f"Predicted price (1200 sqft): PKR {predicted_price[0]:.0f}")
```

10.4 Evaluation Metrics for Regression

Metric	Formula	Interpretation
MAE	Mean of $ y - y_{\text{hat}} $	Average absolute error
MSE	Mean of $(y - y_{\text{hat}})^2$	Penalizes large errors
RMSE	$\sqrt{\text{MSE}}$	Same unit as y
R^2	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	1=perfect, 0=baseline

Review Questions & Case Studies

A. Real-Time Review Questions

1. What is linear regression? Write the equation for multiple linear regression.
2. What is the cost function in linear regression? Why do we minimize it?
3. Explain gradient descent. What happens if the learning rate is too high or too low?
4. Calculate MSE for predictions [3,5,7] vs actuals [2.5,5,6.5].
5. What does R-squared = 0.92 mean? What does R-squared = 0.1 indicate?

B. Case Study

Case Study – Sales Forecasting at a Retail Store: A supermarket chain wants to predict weekly sales based on: advertising spend, day of week, local temperature, and promotional discounts. Using multiple linear regression on 2 years of historical data, the model achieves $R^2 = 0.87$ — explaining 87% of sales variance. This helps managers plan inventory and staffing.

Discussion Questions:

1. Identify the dependent and independent variables in this problem.
2. If the temperature coefficient is 500, what does this mean practically?
3. How would you validate that the regression assumptions are met?

Week 6

Machine Learning – Logistic Regression & Clustering

Lecture 11

Supervised Learning – Logistic Regression

11.1 From Linear to Logistic Regression

While linear regression predicts continuous values, logistic regression predicts probabilities for classification problems. The key difference is the use of the **sigmoid function** which squashes any real number into the range [0, 1].

Definition: Sigmoid Function

$\sigma(z) = 1 / (1 + e^{-z})$ where $z = w_0 + w_1x_1 + \dots + w_nx_n$. Output: probability between 0 and 1. If $\sigma(z) \geq 0.5 \rightarrow$ Class 1, else Class 0.

Example: Student Pass/Fail Prediction

Features: study hours, attendance percentage. Label: pass (1) / fail (0). For a student with $z = 0.8x_1 + 0.3x_2 - 3.5 = 0.8(5) + 0.3(70\%) - 3.5 = 1.01$. $\sigma(1.01) = 1/(1+e^{-1.01}) = 0.73 \rightarrow 73\%$ probability of passing.

```
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix

# Load dataset (breast cancer: malignant=0, benign=1)
data = load_breast_cancer()
X, y = data.data, data.target

# Preprocess
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42)

# Train Logistic Regression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Evaluate
y_pred = model.predict(X_test)
print("Confusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred,
                           target_names=['Malignant', 'Benign']))

# Predict probability for a new sample
prob = model.predict_proba(X_test[:1])
print(f"Malignant: {prob[0][0]:.3f}, Benign: {prob[0][1]:.3f}")
```

11.2 Logistic vs Linear Regression

Feature	Linear Regression	Logistic Regression
Output	Continuous value	Probability (0–1)
Problem Type	Regression	Classification
Function	Linear	Sigmoid
Cost Function	MSE	Log Loss / Cross-Entropy
Example	House price	Spam or not spam

Review Questions & Case Studies

A. Real-Time Review Questions

1. Explain the sigmoid function. Why is it used in logistic regression?
2. What is the decision boundary in logistic regression? How is it determined?
3. Compare logistic regression and linear regression across five dimensions.
4. What is log-loss (cross-entropy)? Why is it preferred over MSE for classification?
5. Can logistic regression handle multi-class classification? Explain the approach.

B. Case Study

Case Study – Bank Loan Default Prediction: A bank wants to predict whether a loan applicant will default (1) or repay (0). Features: credit score, income, employment years, debt-to-income ratio. Logistic regression gives a probability output, e.g., $P(\text{default}) = 0.78 \rightarrow$ reject application. The bank can also adjust the threshold: lowering it to 0.3 reduces risk (more rejections) but fewer loans.

Discussion Questions:

1. Why is logistic regression better than linear regression for this problem?
2. What happens if the bank sets the threshold at 0.3 vs 0.7?
3. How would class imbalance (few defaults) affect the model?

Lecture 12

Unsupervised Learning – K-Means Clustering

12.1 What is Unsupervised Learning?

Definition: Unsupervised Learning

Unsupervised learning algorithms find patterns, structures, or groupings in data without using labeled examples. The algorithm explores the data and discovers hidden structures on its own. Common tasks: Clustering and Dimensionality Reduction.

12.2 K-Means Clustering Algorithm

Definition: K-Means Clustering

K-Means partitions n data points into K clusters. Each point belongs to the cluster with the nearest centroid. Algorithm: (1) Choose K centroids randomly, (2) Assign each point to nearest centroid, (3) Recompute centroid as mean of cluster, (4) Repeat steps 2-3 until convergence (centroids stop moving).

Example: Customer Segmentation

E-commerce store has 1000 customers with features: annual spend and visit frequency. K-Means with $K=3$ identifies: Cluster 1 = High spenders, frequent visitors (VIP), Cluster 2 = Moderate spenders, occasional visitors (Regular), Cluster 3 = Low spenders, rare visitors (Occasional). Each group receives targeted marketing.

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import numpy as np

# Sample customer data: [annual_spend_PKR, visits_per_month]
np.random.seed(42)
customers = np.array([
    [50000, 2], [55000, 3], [48000, 2], [200000, 15], [190000, 14],
    [210000, 16], [12000, 1], [15000, 1], [11000, 1], [100000, 8],
    [95000, 7], [105000, 9], [175000, 13], [180000, 14], [9000, 1]
])

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(customers)

# Apply K-Means with K=3
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X_scaled)

# Results
labels = kmeans.labels_
centroids = scaler.inverse_transform(kmeans.cluster_centers_)
```

```
print("Cluster Assignments:", labels)
print("\nCluster Centroids (Annual Spend, Monthly Visits):")
for i, c in enumerate(centroids):
    print(f"  Cluster {i}: Spend={c[0]:.0f} PKR, Visits={c[1]:.1f}/month")

# Predict cluster for new customer
new_customer = scaler.transform([[80000, 6]])
cluster = kmeans.predict(new_customer)
print(f"\nNew customer belongs to Cluster: {cluster[0]}")
```

12.3 Choosing K – The Elbow Method

The **Elbow Method** plots the Within-Cluster Sum of Squares (WCSS) vs K. The 'elbow' point where the WCSS decrease slows down significantly is the optimal K. Beyond this point, adding more clusters provides diminishing returns.

Review Questions & Case Studies

A. Real-Time Review Questions

1. Explain the K-Means algorithm step by step with a numerical example.
2. What is the Elbow Method? How does it help in choosing the optimal K?
3. What are the limitations of K-Means clustering? When does it fail?
4. Compare K-Means (unsupervised) with K-Nearest Neighbors (supervised).
5. Apply K-Means manually: 6 points in 2D space, K=2. Show two iterations.

B. Case Study

Case Study – Spotify Music Clustering: Spotify uses clustering to group songs and users. Songs are clustered based on audio features: tempo, energy, danceability, acousticness, valence. K-Means (K=10) creates 10 music 'moods': energetic pop, calm acoustic, heavy metal, classical, etc. Users are then matched to playlists based on which cluster their listening history falls into.

Discussion Questions:

1. Why is clustering appropriate here rather than classification?
2. How would Spotify determine the optimal number of clusters K?
3. What happens if two very different songs end up in the same cluster?

Week 7 Machine Learning – Decision Trees & Association Rules

Lecture 13 Decision Trees & ID3 Algorithm

13.1 What is a Decision Tree?

Definition: Decision Tree

A Decision Tree is a flowchart-like tree structure where each internal node tests a feature, each branch represents the test outcome, and each leaf node gives a class label or value. It mimics human decision making and is highly interpretable.

13.2 Key Concepts

Entropy: Measures impurity/disorder in a dataset: $H(S) = -\sum p_i \times \log_2(p_i)$. $H=0$ means pure; $H=1$ means maximum impurity.

Information Gain: Reduction in entropy after splitting on a feature: $IG = H(\text{parent}) - \sum (|S_v|/|S|) \times H(S_v)$. Higher IG $\rightarrow$ better split.

Gini Impurity: Alternative to entropy: $Gini = 1 - \sum p_i^2$. Used by CART algorithm.

Pruning: Removing branches that have little predictive power to prevent overfitting.

13.3 ID3 Algorithm

ID3 (Iterative Dichotomiser 3) builds a decision tree by recursively selecting the feature with the highest Information Gain at each step.

Example: Play Tennis Decision

Dataset: 14 days, features: Outlook (Sunny/Overcast/Rain), Temperature, Humidity, Wind. Label: PlayTennis (Yes/No). 9 Yes, 5 No. $H(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.940$ bits. Outlook has IG = 0.246 (highest) $\rightarrow$ chosen as root. Overcast subtree: all Yes $\rightarrow$ leaf node.

```
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
```

```
# Train Decision Tree using entropy (ID3-like)
dt = DecisionTreeClassifier(criterion='entropy', max_depth=4, random_state=42)
dt.fit(X_train, y_train)

# Evaluate
y_pred = dt.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")

# Print tree structure
tree_rules = export_text(dt, feature_names=iris.feature_names)
print("Decision Tree Rules:")
print(tree_rules[:800]) # Print first portion

# Feature importances
print("\nFeature Importances:")
for name, imp in zip(iris.feature_names, dt.feature_importances_):
    print(f" {name}: {imp:.4f}")
```

13.4 Advantages and Disadvantages

Advantages	Disadvantages
Easy to understand and visualize	Prone to overfitting (deep trees)
No feature scaling required	Sensitive to small data changes
Handles both numerical and categorical data	Biased toward features with more levels
Can handle missing values	Not ideal for continuous outputs
Fast training and prediction	Single tree can be unstable

Review Questions & Case Studies

A. Real-Time Review Questions

1. What is entropy in the context of Decision Trees? Calculate entropy for a set with 6 yes and 4 no.
2. Explain the ID3 algorithm. What feature does it select at each step?
3. What is information gain? Calculate IG for Outlook attribute in the tennis example.
4. What is pruning? Explain pre-pruning vs post-pruning with examples.
5. Compare Decision Tree with Naive Bayes for a spam detection task.

B. Case Study

Case Study – Loan Approval System (HDFC Bank): HDFC Bank uses a decision tree model to automate initial loan screening. The tree asks: Is credit score > 700? → Is income > PKR 50,000/month? → Is debt-to-income ratio < 40%? → Approve. The tree was trained on 50,000 historical loan applications with features including employment type, credit history, collateral, and age. Accuracy on test set: 89%.

Discussion Questions:

1. What criterion (entropy or Gini) would you use to build this tree?
2. What max_depth would prevent overfitting while maintaining accuracy?
3. What are the fairness/bias concerns with using AI for loan decisions?

Lecture 14

Association Rules for Decision Making

14.1 What are Association Rules?

Definition: Association Rule Mining

Association rule mining discovers interesting relationships between variables in large datasets. Rule format: $X \rightarrow Y$ (if X then Y). Example: $\{\text{Bread, Butter}\} \rightarrow \{\text{Milk}\}$ means customers who buy bread and butter also buy milk.

14.2 Key Metrics

Metric	Formula	Meaning
Support	$\text{freq}(X \cup Y) / N$	How often rule appears in dataset
Confidence	$\text{freq}(X \cup Y) / \text{freq}(X)$	How often rule is correct
Lift	$\text{Confidence} / \text{Support}(Y)$	Strength vs random chance (>1 = useful)

Example: Supermarket Basket Analysis

1000 transactions. Bread bought 400 times. Milk bought 500 times. Both bought together 300 times. $\text{Support}(\{\text{Bread, Milk}\}) = 300/1000 = 0.30$ (30%). $\text{Confidence}(\text{Bread} \rightarrow \text{Milk}) = 300/400 = 0.75$ (75%). $\text{Lift} = 0.75 / 0.50 = 1.5 \rightarrow$ Buying bread increases milk purchase probability by 50%.

14.3 Apriori Algorithm

The Apriori algorithm efficiently finds all frequent itemsets by using the principle: if an itemset is frequent, all its subsets must also be frequent. It prunes infrequent candidates early to avoid scanning the entire dataset.

```
# Install: pip install mlxtend
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
import pandas as pd

# Sample transaction data
transactions = [
    ['Bread', 'Milk', 'Butter'],
    ['Bread', 'Milk'],
    ['Milk', 'Butter', 'Eggs'],
    ['Bread', 'Butter'],
    ['Bread', 'Milk', 'Eggs'],
    ['Milk', 'Eggs'],
    ['Bread', 'Milk', 'Butter', 'Eggs'],
    ['Bread', 'Butter'],
]

# Encode transactions
te = TransactionEncoder()
```



```
te_array = te.fit_transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)

# Find frequent itemsets (min support = 0.4)
frequent_items = apriori(df, min_support=0.4, use_colnames=True)
print("Frequent Itemsets:")
print(frequent_items)

# Generate association rules (min confidence = 0.6)
rules = association_rules(frequent_items, metric="confidence", min_threshold=0.6)
rules_sorted = rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
print("\nAssociation Rules:")
print(rules_sorted.sort_values('lift', ascending=False).to_string(index=False))
```

Review Questions & Case Studies

A. Real-Time Review Questions

1. Define support, confidence, and lift for association rules with formulas and examples.
2. Given: 500 transactions, {Tea,Biscuit} appears 100 times, {Tea} 200 times, {Biscuit} 150. Calculate all three metrics.
3. Explain the Apriori algorithm. What is the 'anti-monotone' property?
4. What is the difference between association rules and classification?
5. Give three real-world applications of association rule mining beyond retail.

B. Case Study

Case Study – Amazon's 'Customers Also Bought': Amazon's recommendation engine uses association rule mining on billions of transactions. If $P(\text{laptop, mouse}) \rightarrow \text{keyboard}$ has high lift and confidence, Amazon prominently displays keyboards to laptop buyers. This 'collaborative filtering via association rules' is estimated to drive 35% of Amazon's total revenue. The challenge is scale: millions of products require distributed computing (Hadoop/Spark) to mine.

Discussion Questions:

1. What minimum support and confidence thresholds would you set for Amazon's scale?
2. Why is lift more meaningful than confidence alone for recommendations?
3. How can negative associations ($X \rightarrow \text{NOT } Y$) be useful for cross-selling?

Week 8 Neural Networks & Reinforcement Learning

Lecture 15 Neural Networks & Perceptron

15.1 Biological Inspiration

Artificial Neural Networks (ANNs) are inspired by the human brain. The brain contains ~86 billion neurons connected by synapses. Each neuron receives signals, processes them, and fires if the signal is strong enough. ANNs model this with mathematical nodes connected by weighted edges.

15.2 The Perceptron

Definition: Perceptron

The perceptron is the simplest neural network unit, invented by Frank Rosenblatt (1957). It takes input features, multiplies by weights, sums them, adds a bias, and passes through an activation function: $\text{output} = f(w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b)$. The perceptron can only classify linearly separable data.

Example: AND Gate with Perceptron

Inputs: $x_1, x_2 \in \{0,1\}$. Weights: $w_1=w_2=1$. Bias: $b=-1.5$. AND(0,0): $0+0-1.5=-1.5 \rightarrow \text{step}(-1.5)=0$. AND(1,1): $1+1-1.5=0.5 \rightarrow \text{step}(0.5)=1$. AND(1,0): $1+0-1.5=-0.5 \rightarrow \text{step}(-0.5)=0$. Perceptron correctly implements AND gate.

15.3 Activation Functions

Function	Formula	Range	Use Case
Step	1 if $z \geq 0$ else 0	[0,1]	Binary classification (perceptron)
Sigmoid	$1/(1+e^{-z})$	(0,1)	Output layer, binary classification
Tanh	$(e^z - e^{-z}) / (e^z + e^{-z})$	(-1,1)	Hidden layers
ReLU	$\max(0, z)$	$[0, \infty)$	Deep networks, hidden layers
Softmax	$e^{z_i} / \sum e^{z_j}$	(0,1)	Multi-class output layer

15.4 Multi-Layer Perceptron (Deep Neural Network)

A Multi-Layer Perceptron (MLP) has: (1) Input Layer – receives features, (2) Hidden Layer(s) – learn complex representations, (3) Output Layer – produces final predictions. Training uses **Backpropagation** and **Gradient Descent** to minimize loss.

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
```

```

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# Generate non-linear dataset (moons)
X, y = make_moons(n_samples=300, noise=0.2, random_state=42)

# Split and scale
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build MLP: 2 hidden layers with 64 and 32 neurons
mlp = MLPClassifier(
    hidden_layer_sizes=(64, 32), # 2 hidden layers
    activation='relu',           # ReLU activation
    solver='adam',               # Adam optimizer
    max_iter=500,
    random_state=42
)
mlp.fit(X_train, y_train)

# Evaluate
y_pred = mlp.predict(X_test)
print(f"MLP Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"Layers: {mlp.n_layers_} total")
print(f"Iterations: {mlp.n_iter_}")
print(f"Loss: {mlp.loss_: .6f}")

```

15.5 Backpropagation

Backpropagation is the algorithm for training neural networks. Steps: (1) Forward Pass: compute predictions; (2) Compute Loss: measure error; (3) Backward Pass: calculate gradient of loss w.r.t. each weight using chain rule; (4) Update Weights: $w = w - \alpha \times \text{gradient}$. Repeated for many epochs until convergence.

Review Questions & Case Studies

A. Real-Time Review Questions

1. What is a perceptron? Explain its components: inputs, weights, bias, activation function.
2. Implement an AND gate and OR gate using a perceptron (show weights manually).
3. What is backpropagation? Explain the forward pass and backward pass.
4. Compare ReLU and Sigmoid activation functions. Why is ReLU preferred in deep networks?
5. What is the vanishing gradient problem? Which activation function helps solve it?

B. Case Study

Case Study – Image Recognition at PIA (Pakistan International Airlines): A security system at Islamabad International Airport uses a convolutional neural network (CNN) trained on 500,000 images to detect prohibited items in baggage. The network has: Input layer (image pixels), 4 convolutional layers, 2 fully connected layers, Output: 20 classes (knife, gun, liquid, laptop, etc.). Accuracy: 96.3%. Processing time: 0.3 seconds per bag.

Discussion Questions:

1. Why is a neural network better than a simple decision tree for image detection?
2. What activation function would you use in the output layer for 20 classes?
3. How would you handle a new prohibited item not in the training data?

Lecture 16 Reinforcement Learning

16.1 What is Reinforcement Learning?

Definition: Reinforcement Learning (RL)

Reinforcement Learning is a type of machine learning where an agent learns to make decisions by interacting with an environment. The agent receives rewards for correct actions and penalties for wrong ones, and learns a policy that maximizes cumulative reward. Unlike supervised learning, there are no labels — only feedback signals.

16.2 Key Components of RL

Component	Symbol	Description
Agent	π	The learner/decision maker
Environment	E	What the agent interacts with
State	s	Current situation of agent in environment
Action	a	What the agent can do
Reward	r	Feedback signal from environment
Policy	$\pi(s)$	Strategy: action to take in each state
Value Function	V(s)	Expected cumulative future reward from state s
Q-Function	Q(s,a)	Expected reward for action a in state s

16.3 The RL Loop

Agent observes State (s) → Chooses Action (a) based on Policy → Environment transitions to new State (s') → Environment gives Reward (r) → Agent updates Policy → Repeat until goal achieved or episode ends.

16.4 Q-Learning Algorithm

Definition: Q-Learning

Q-Learning is a model-free RL algorithm that learns the Q-value $Q(s,a)$: the expected cumulative reward for taking action a in state s. Update rule: $Q(s,a) \leftarrow Q(s,a) + \alpha \times [r + \gamma \times \max_{a'} Q(s',a') - Q(s,a)]$ where α = learning rate, γ = discount factor (0 to 1).

```
import numpy as np

# Simple Grid World: 4x4 grid, goal at (3,3), hole at (1,1) and (2,3)
# States: 0-15 (flattened grid), Actions: 0=Up,1=Down,2=Left,3=Right

def create_env():
```

```

"""Returns reward matrix for grid world."""
rewards = np.full((16, 4), -1.0) # -1 for each step
rewards[15, :] = 10.0           # Goal state (3,3): reward +10
rewards[5, :] = -10.0           # Hole (1,1): penalty -10
rewards[11, :] = -10.0          # Hole (2,3): penalty -10
return rewards

def q_learning(episodes=500, alpha=0.1, gamma=0.9, epsilon=0.2):
    Q = np.zeros((16, 4))        # Q-table: 16 states, 4 actions
    rewards_arr = create_env()

    for ep in range(episodes):
        state = 0                 # Start at top-left (0,0)
        total_reward = 0

        for step in range(100):   # Max 100 steps per episode
            # Epsilon-greedy policy
            if np.random.rand() < epsilon:
                action = np.random.randint(4) # Explore
            else:
                action = np.argmax(Q[state])   # Exploit

            # Simulate transition (simple: action moves state index)
            next_state = min(15, max(0, state + [-4, 4, -1, 1][action]))
            reward = rewards_arr[state, action]
            total_reward += reward

            # Q-update
            Q[state, action] += alpha * (
                reward + gamma * np.max(Q[next_state]) - Q[state, action])

            state = next_state
            if state == 15:        # Reached goal
                break

        if (ep + 1) % 100 == 0:
            print(f"Episode {ep+1}: Total Reward = {total_reward:.1f}")

    print("\nOptimal Policy (best action per state):")
    actions = ['Up', 'Down', 'Left', 'Right']
    for i in range(16):
        print(f"State {i:2d}: {actions[np.argmax(Q[i])]}")
    return Q

Q_table = q_learning()

```

16.5 Types of Reinforcement Learning

Type	Description	Example
Model-Free	Learns directly from interaction	Q-Learning, SARSA
Model-Based	Learns environment model first	Dyna-Q, World Models
Policy Gradient	Directly optimizes policy	REINFORCE, PPO
Actor-Critic	Combines policy + value	A3C, SAC
Deep RL	RL + Deep Neural Networks	DQN, AlphaGo, ChatGPT (RLHF)

16.6 RL vs Supervised Learning

Aspect	Supervised Learning	Reinforcement Learning
Input	Labeled (X,y) pairs	State, action, reward
Feedback	Immediate correct label	Delayed reward signal
Goal	Predict output	Maximize cumulative reward
Data	Static dataset	Dynamic environment interaction
Example	Email spam filter	Game playing, robotics

Review Questions & Case Studies

A. Real-Time Review Questions

1. Define Reinforcement Learning. Explain its key components with a chess-playing agent example.
2. What is a Q-table? How does Q-Learning update Q-values using the Bellman equation?
3. Explain the exploration vs exploitation tradeoff. What is epsilon-greedy?
4. Compare Reinforcement Learning with Supervised Learning across four dimensions.
5. What is Deep Q-Network (DQN)? How did it achieve superhuman performance in Atari games?

B. Case Study

Case Study – DeepMind AlphaGo: In March 2016, Google DeepMind's AlphaGo defeated 18-time world champion Lee Sedol in the board game Go — considered the most complex game ever created (10^{170} possible positions). AlphaGo used deep reinforcement learning: a policy network (choosing moves) and a value network (evaluating positions) trained by playing millions of games against itself (self-play RL). The agent received +1 reward for winning, -1 for losing.

Discussion Questions:

1. Why was Go considered a far harder challenge for AI than chess?
2. How did AlphaGo use both supervised learning and reinforcement learning?
3. What are the ethical implications of AI systems surpassing human expertise?
4. How could similar RL techniques be applied to traffic optimization in Lahore?

Course Summary

Week	Lectures	Core Topic	Key Algorithms / Concepts
W1	L1–L2	Intro to AI	History, Agents, PEAS, Applications
W2	L3–L4	Problem Solving	State Space, BFS, DFS
W3	L5–L6	Search + Intro ML	A*, Greedy, ML Pipeline
W4	L7–L8	Supervised Learning	Types, Naive Bayes, Bayes Theorem
W5	L9–L10	Metrics + Regression	Confusion Matrix, MSE, Linear Regression
W6	L11–L12	Classification + Clustering	Logistic Regression, K-Means
W7	L13–L14	Trees + Rules	ID3, Decision Trees, Apriori
W8	L15–L16	NN + RL	Perceptron, Backprop, Q-Learning

Recommended Textbook: Artificial Intelligence: A Modern Approach (Russell & Norvig, 4th Edition)
— Chapters referenced in each lecture.

Supplementary: Pattern Recognition and Machine Learning (Bishop) | Hands-On Machine Learning with Scikit-Learn (Géron, 3rd Edition).

Important Note: These notes are prepared exclusively for students of **RIC Harbanspura Campus**.
For queries, contact your instructor **Hafiz Muhammad Usman** during scheduled office hours.